

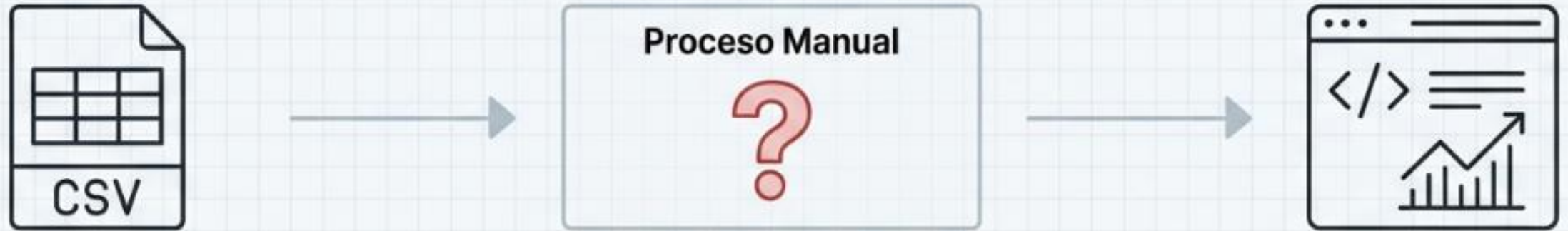
AquaSenseCloud

Diseño e implementación de un pipeline de datos serverless para el análisis de información ambiental en AWS.

Proyecto Final: Infraestructura para la Computación de Altas Prestaciones
Curso: 2025/2026

Javier Hernández Rosique
Álvaro Espejo Martínez

El Desafío: De Datos Crudos a Información Útil

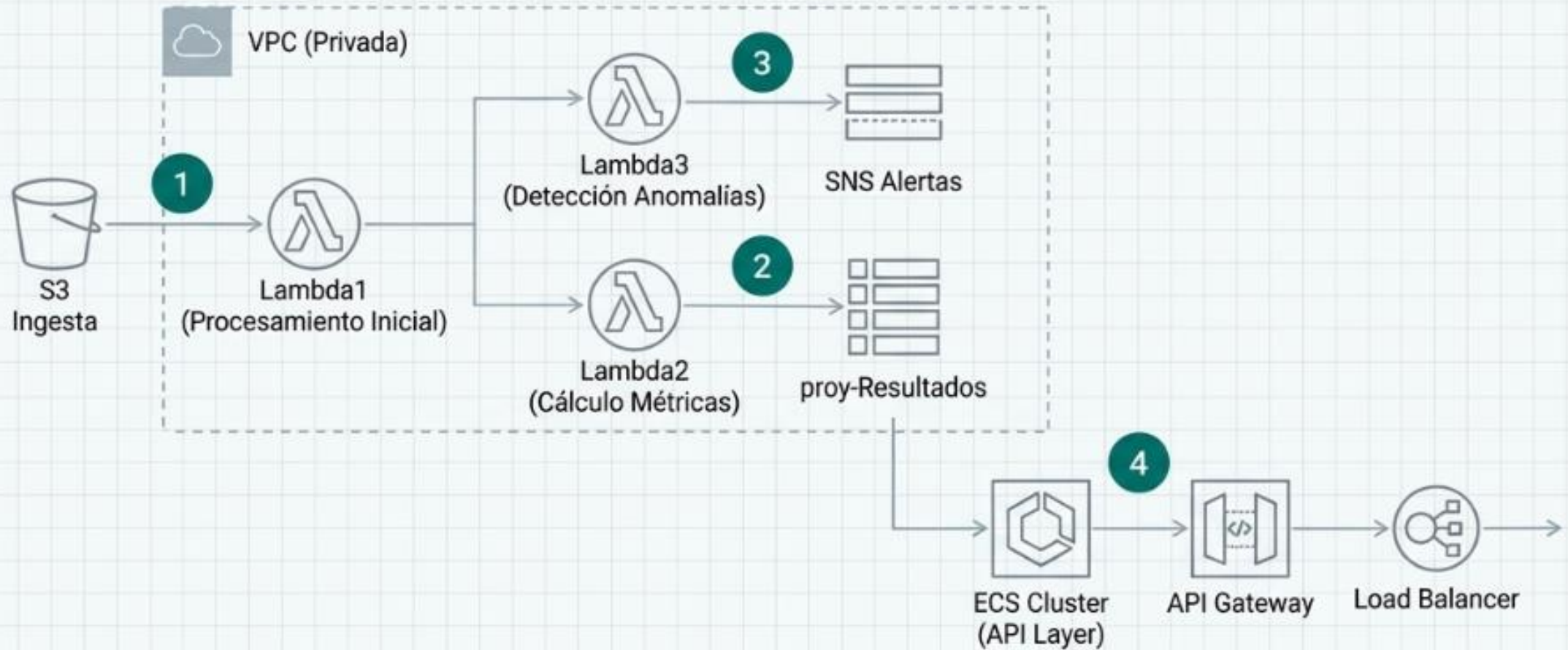


El punto de partida es un conjunto de datos ambientales generados por balizas en el Mar Menor, almacenados en ficheros CSV. El objetivo es transformar este proceso manual en un sistema automatizado, eficiente y escalable.

Requisitos Clave

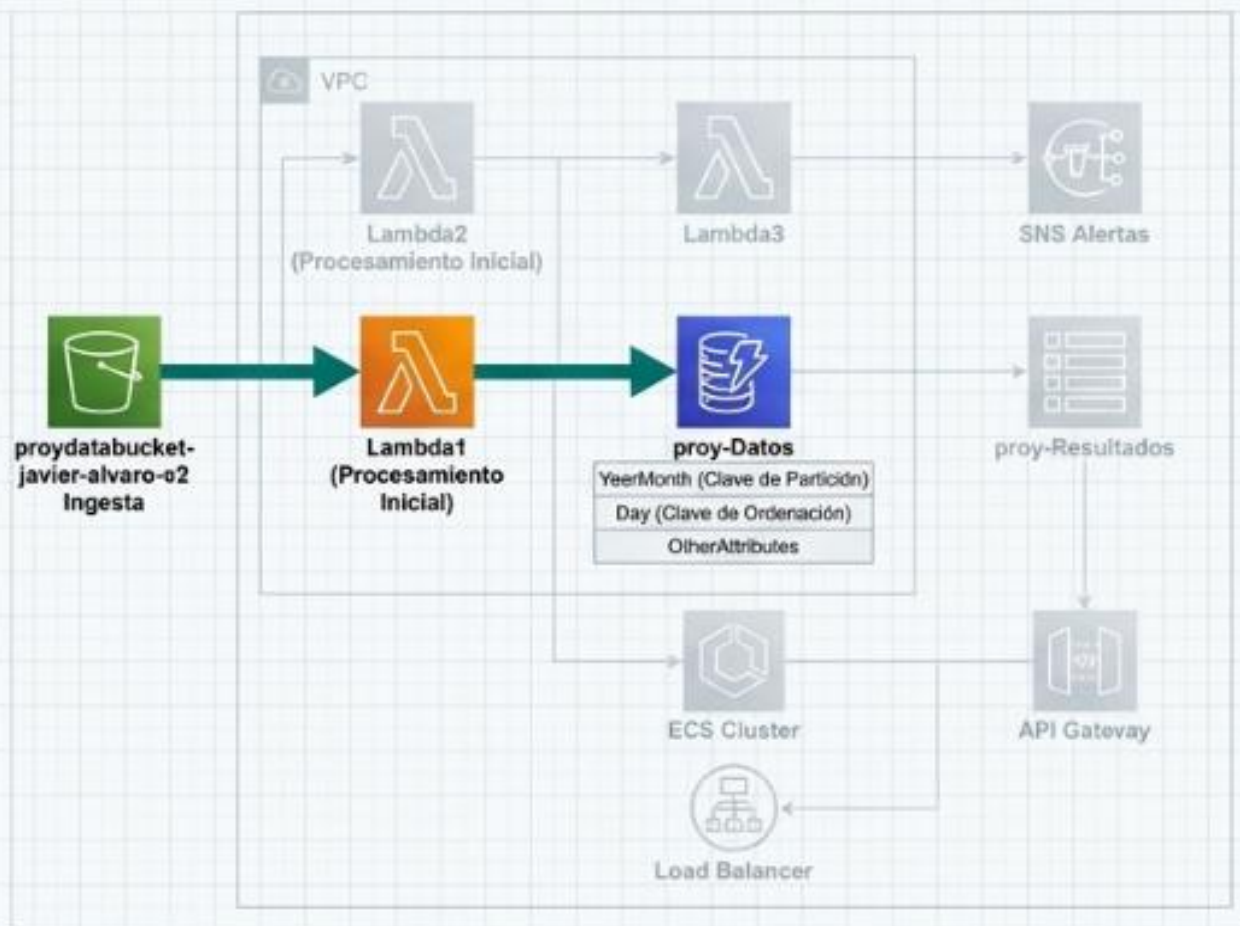
1. **Ingesta Automática:** Disparar el procesamiento al recibir nuevos datos.
2. **Procesamiento Eficiente:** Calcular métricas mensuales y detectar anomalías.
3. **Acceso Fiable:** Exponer los resultados a través de una API de consulta.
4. **Infraestructura Resiliente:** Garantizar alta disponibilidad y seguridad.

El Blueprint: Nuestra Arquitectura de Solución en AWS



Esta es la hoja de ruta de nuestro sistema. En las siguientes diapositivas, realizaremos un recorrido guiado siguiendo el viaje de un dato, desde su ingesta en S3 hasta su consulta final a través de la API.

Etapa 1: Ingesta y Normalización de Datos

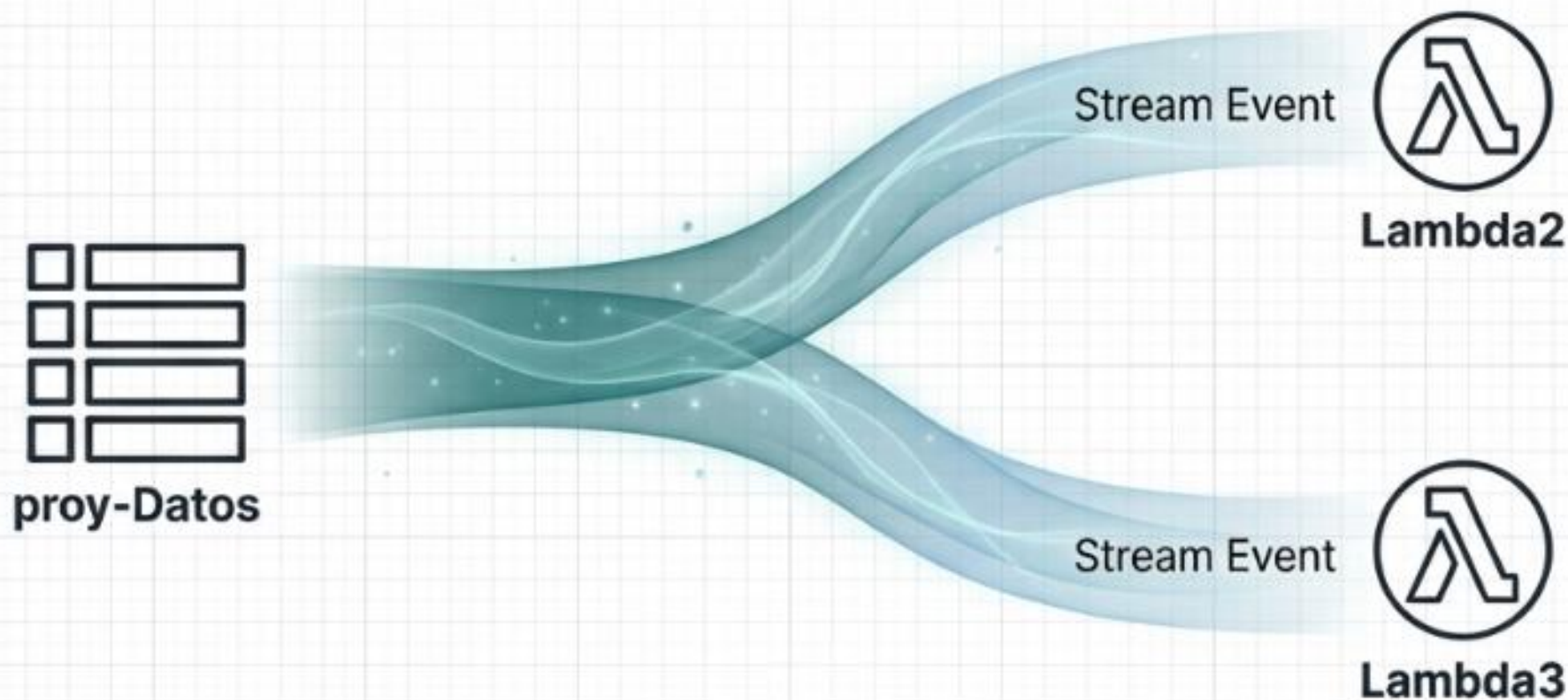


Flujo del Proceso

1. **Activación:** Un nuevo fichero CSV se sube al bucket S3 `proydatabucket-javier-alvaro-v2`.
2. **Desencadenante:** El evento S3 invoca automáticamente la función `Lambda1`.
3. **Acción de Lambda1:**
 - Lee y procesa los registros del CSV.
 - Utiliza un `batch writer` para una inserción masiva eficiente y consistente en DynamoDB.
 - Normaliza el campo de fecha `aaaa-mm-dd` en dos atributos clave:
 - `YearMonth` (Clave de Partición)
 - `Day` (Clave de Ordenación)
 - Los datos normalizados se almacenan en la tabla `proy-Datos`.

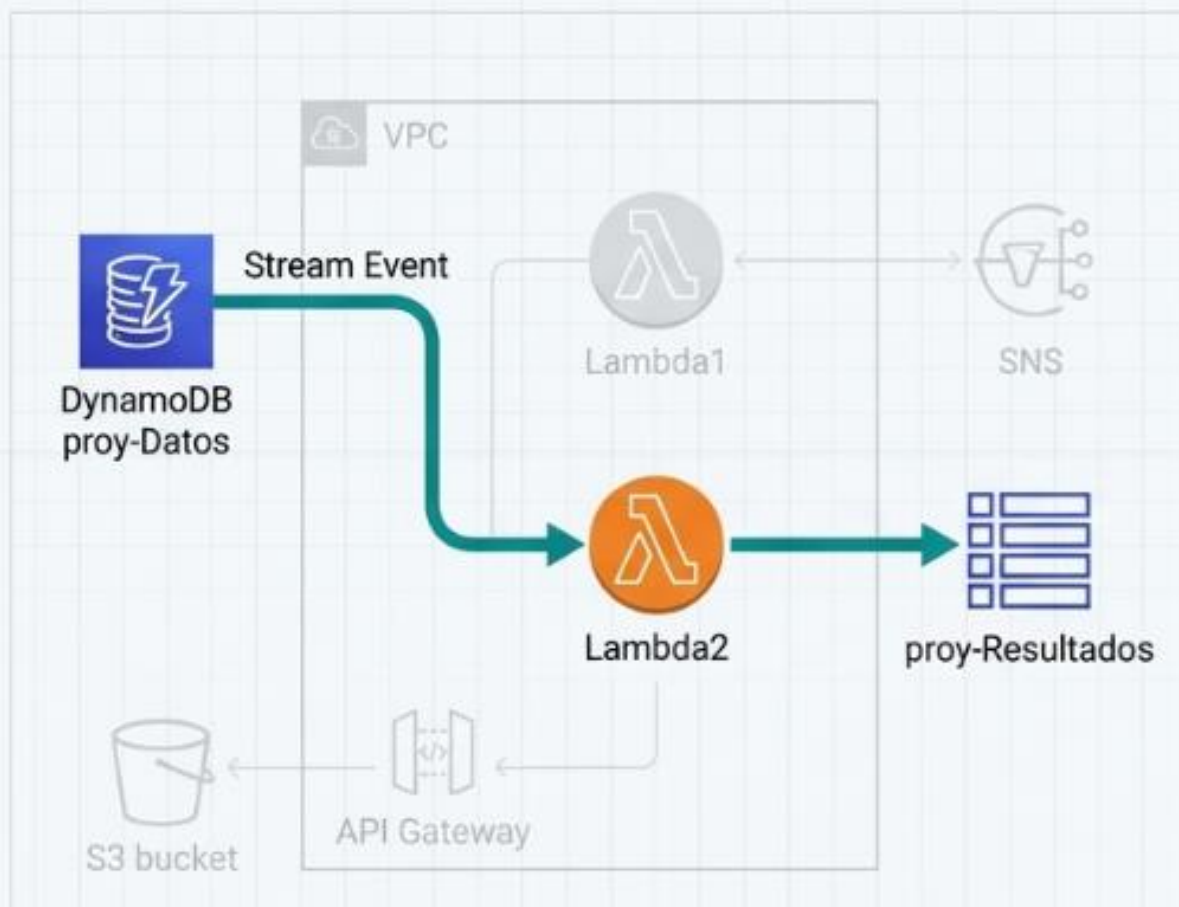
"Esta decisión de modelado responde a buenas prácticas de diseño en DynamoDB, donde una única clave de partición que agrupa todos los registros de un mes permite resolver las consultas con una sola operación eficiente (Query) en lugar de escaneos más costosos."

El Núcleo Reactivo: Coordinación a través de DynamoDB Streams



La tabla `proy-Datos` está configurada con DynamoDB Streams. Cada inserción o modificación de un registro genera un evento de cambio. Estos eventos actúan como desencadenantes para las siguientes funciones Lambda, creando un pipeline encadenado y completamente automatizado. Este enfoque elimina la necesidad de sondeos (polling) y permite un procesamiento incremental en tiempo real.

Etapa 2: El Motor de Procesamiento Incremental

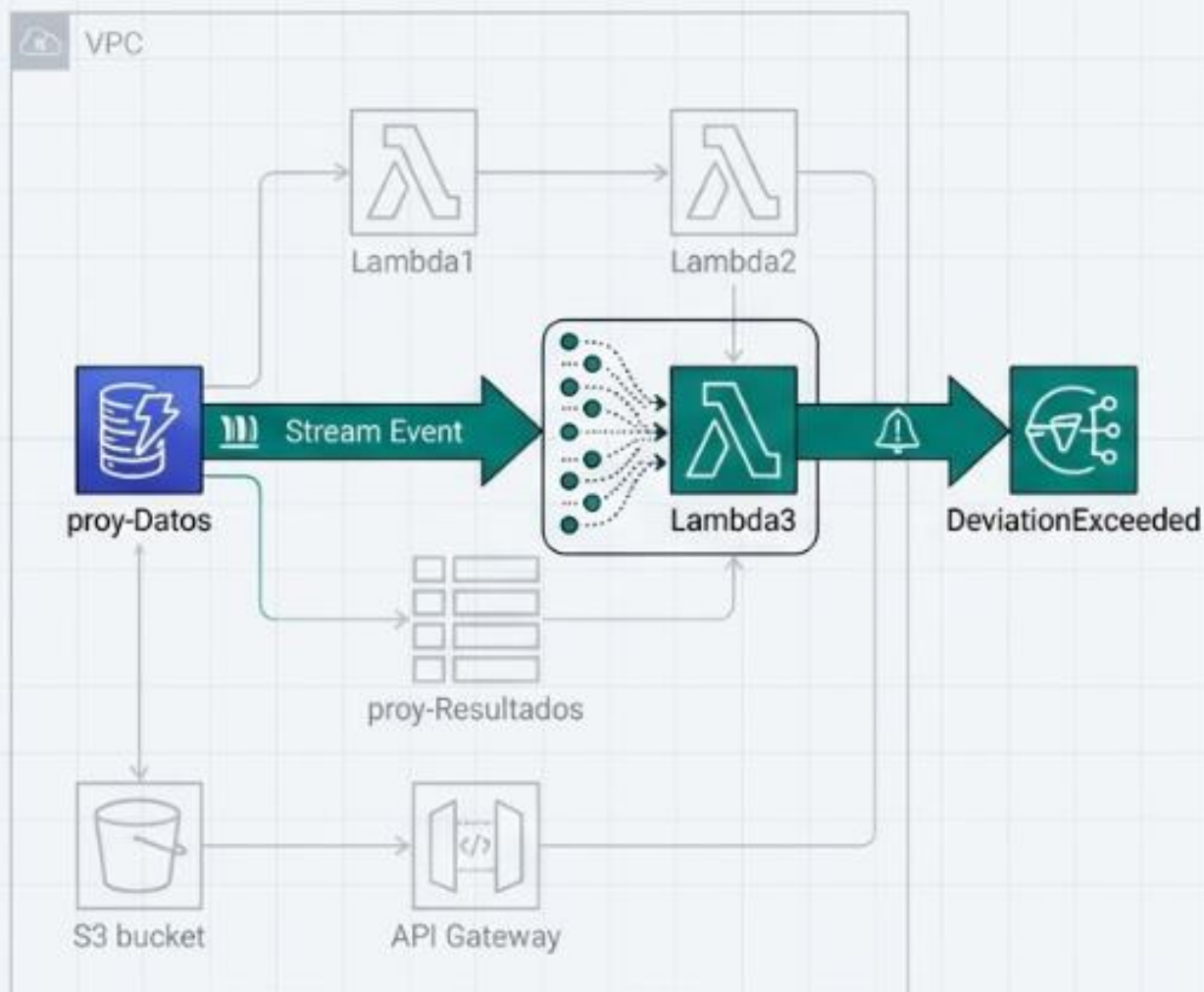


Lógica de `Lambda2`

- Activada por un **evento de Stream**, recibe un lote de cambios.
- Para cada cambio, recupera todos los registros del mes afectado (`Query` por `YearMonth`), así como datos del mes anterior y, si es necesario, del siguiente.
- Calcula de forma incremental las siguientes métricas:
 - Temperatura media del mes.
 - Desviación estándar de las muestras.
 - Diferencia entre la temperatura máxima del mes y la del mes anterior.
- Los resultados actualizados se almacenan en la tabla **`proy-Resultados`**.

"El uso de DynamoDB Streams nos permite adoptar un enfoque reactivo e incremental, en el que solo procesamos los elementos modificados en lugar de recalcular todo el histórico. Esto reduce de forma significativa el consumo de recursos."

Etapa 3: Sistema de Alerta Temprana de Anomalías

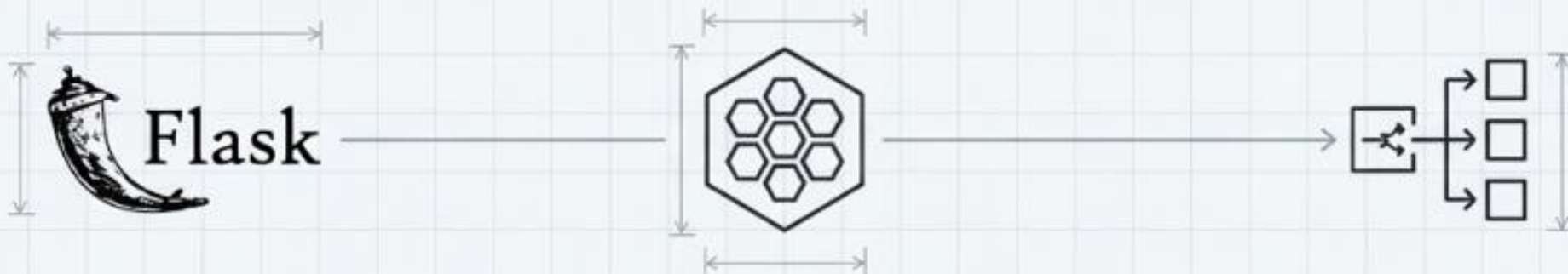


Lógica de `Lambda3`

- Se activa también a través del Stream de `proy-Datos`.
- Analiza los eventos en **lotes** (hasta **100** elementos) para detectar anomalías.
- **Condición de Alerta:** Si algún registro presenta una desviación (`Desviaciones`) superior al umbral de **0.5**.
- **Acción:**
 1. Reúne todas las fechas afectadas en un único mensaje.
 2. Publica el aviso en el tópico de Amazon SNS `DeviationExceeded`.
 3. SNS distribuye la alerta por correo electrónico a los suscriptores.

"La decisión de procesar los eventos en lotes responde a la necesidad de limitar el número de correos generados, evitando saturar a los usuarios con alertas repetitivas."

El Destino Final: Acceso a Resultados vía API Flask



Descripción del Servicio

- Una aplicación Flask, desplegada en contenedores Docker sobre un clúster ECS.
- Actúa como la interfaz de consulta, leyendo los datos **ya procesados** de la tabla `proy-Resultados`.
- Las consultas son altamente eficientes al usar la clave de partición **YearMonth**.

Rutas de la API y Ejemplo de Respuesta

Rutas disponibles: `/maxdiff`, `/sd`, `/temp`
(requieren parámetros `year` y `month`).

Ejemplo de Consulta:

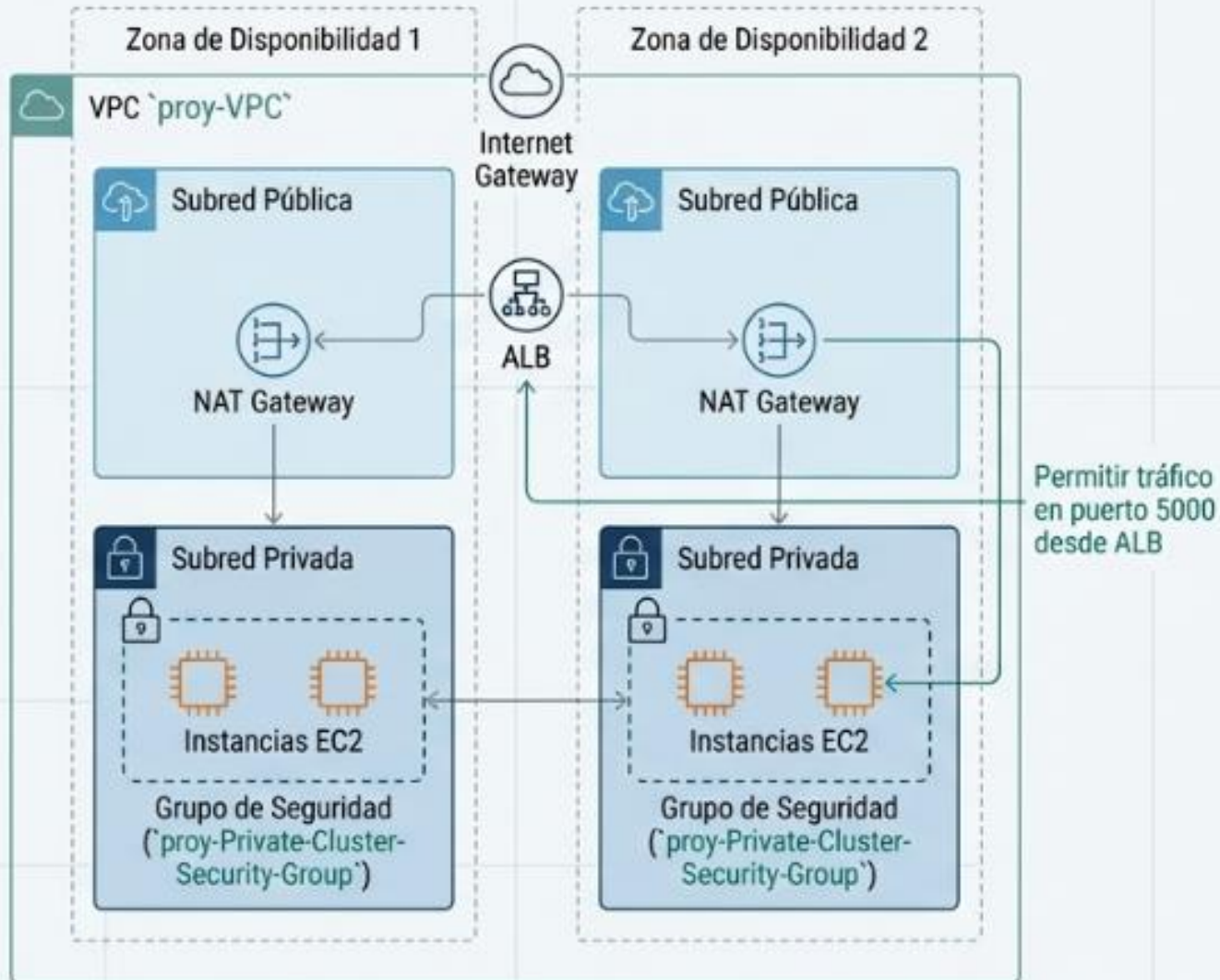
`http://<ALB_DNS>:5000/maxdiff?month=1&year=2023`

Ejemplo de Respuesta JSON:

```
{
  "Date": "2023-01",
  "Maxdiff": 2.75,
  "ip": "10.0.1.152"
}
```

La respuesta incluye la IP privada de la instancia EC2, facilitando la verificación del balanceo de carga.

La Base: Una Red Segura y de Alta Disponibilidad

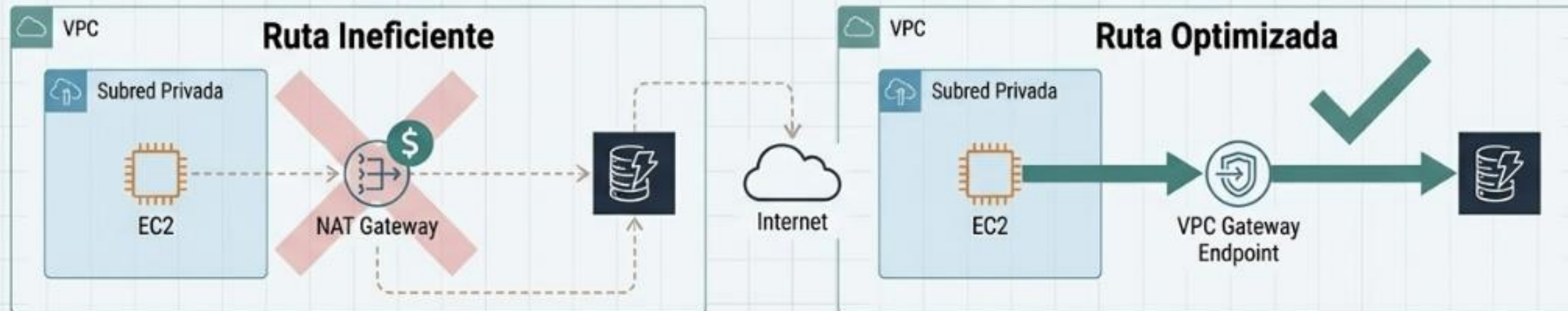


Pilares de la Infraestructura de Red

- **Aislamiento:** Todo desplegado dentro de una VPC (**proy-VPC**) para máxima seguridad.
- **Alta Disponibilidad:** Uso de 2 subredes públicas y 2 privadas distribuidas en dos Zonas de Disponibilidad distintas.
- **Control de Acceso:**
 - **Internet Gateway** para el tráfico de las subredes públicas.
 - **NAT Gateway** para el acceso a internet de las instancias privadas (ej. registro en el clúster ECS).
 - Grupos de Seguridad restrictivos: El clúster ECS (**proy-Private-Cluster-Security-Group**) solo acepta tráfico del balanceador de carga en el puerto 5000.

Optimización de Costes y Rendimiento con VPC Endpoints

Para evitar costes de transferencia de datos a través del NAT Gateway y reducir la latencia, todo el tráfico interno hacia servicios de AWS se enruta de forma optimizada.



Detalles Técnicos

- Se utilizan **VPC Gateway Endpoints** para S3 (`S3Endpoint`) y DynamoDB (`DynamoDBEndpoint`).
- El tráfico de datos permanece dentro de la red privada de AWS.

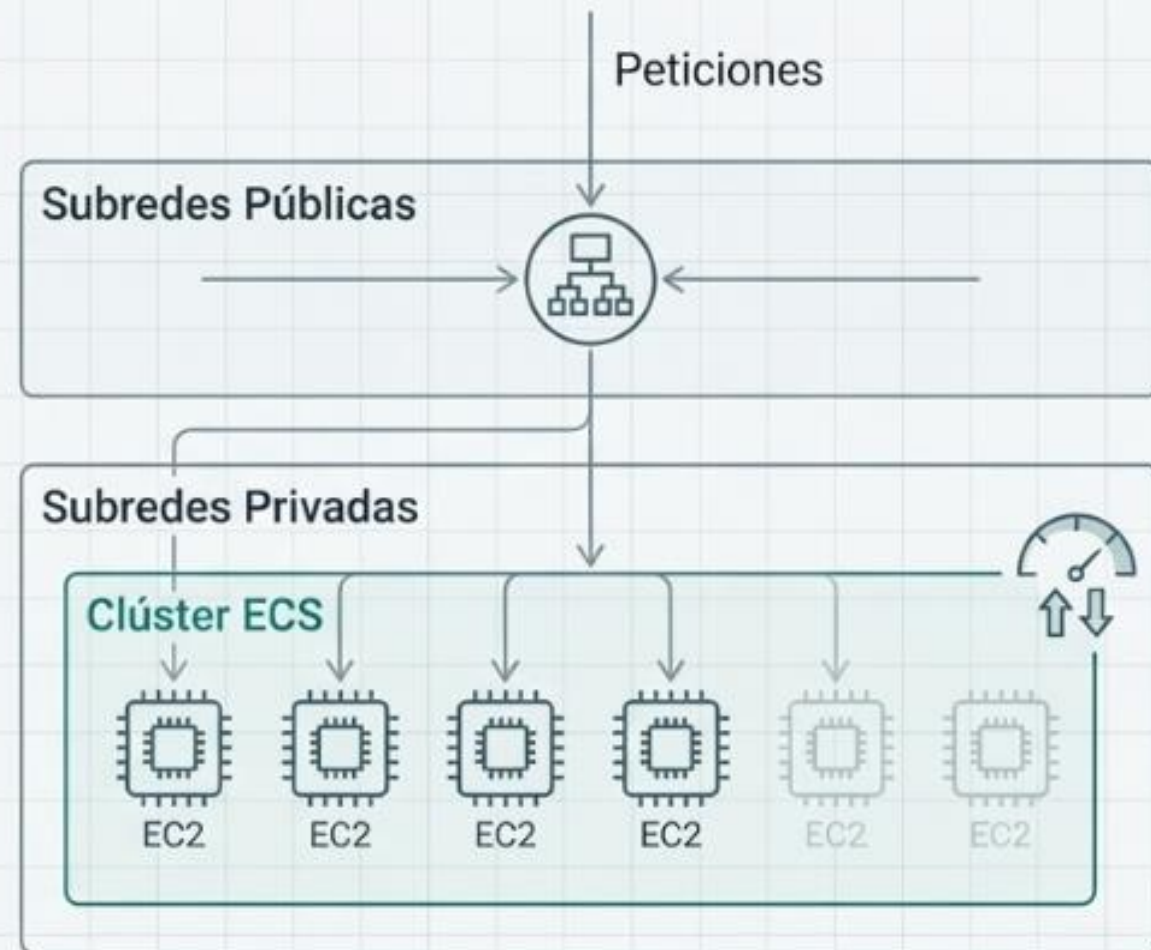
Beneficios

- **Reducción de latencia.**
- **Eliminación de costes** de procesamiento de datos del NAT Gateway.
- **Mejora de la seguridad** al no exponer el tráfico a internet.

El Hogar de la Aplicación: Un Servicio de Contenedores Escalable

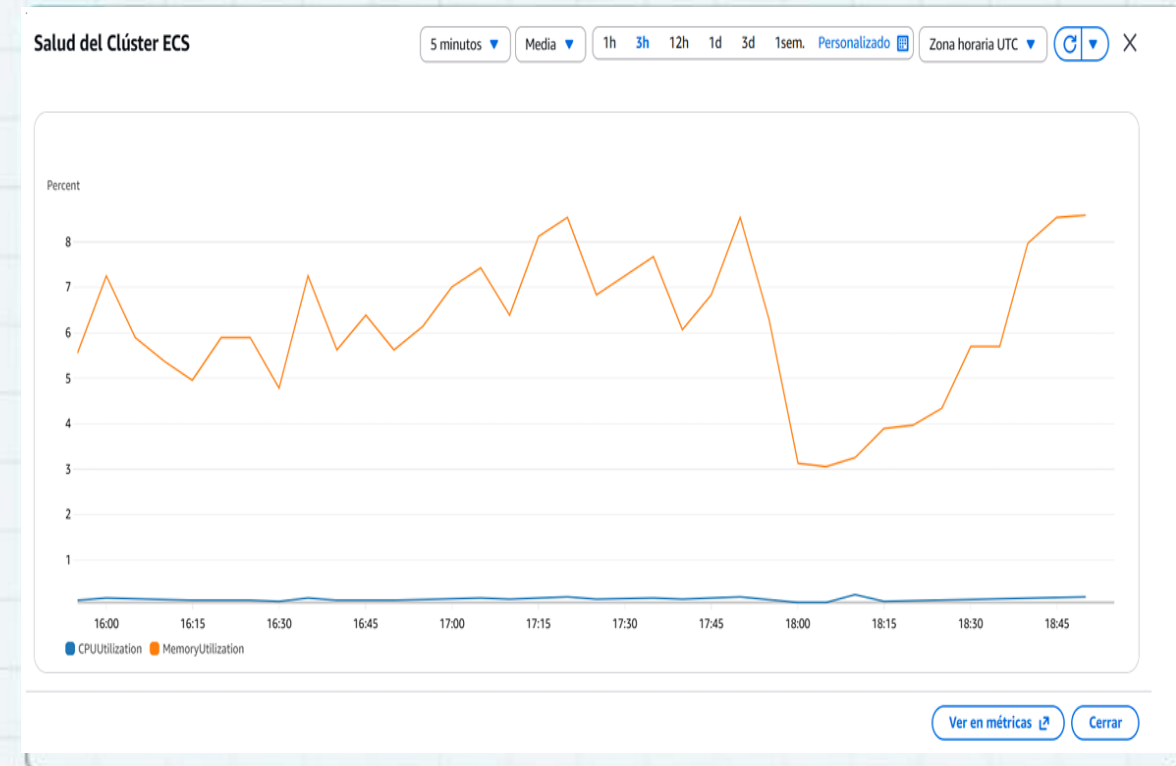
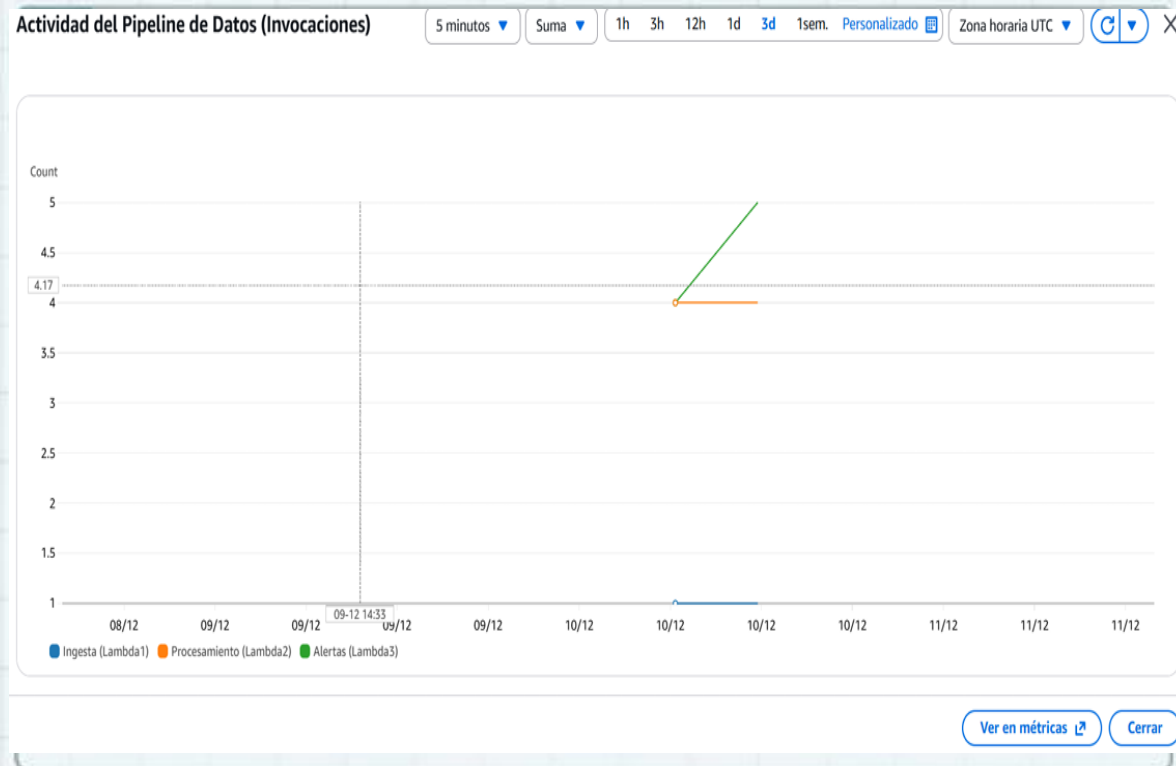
Componentes Clave

- **Imagen Docker:** La aplicación Flask se empaqueta y almacena en un repositorio ECR (`proy-repo`).
- **Clúster ECS:** Orquesta la ejecución de los contenedores (`proy-ecs-cluster`) sobre instancias EC2.
- **Balanceo de Carga:** Un Application Load Balancer (`flask-load-balancer`) distribuye las peticiones entrantes, actuando como único punto de acceso.
- **Auto Scaling:** El clúster está configurado para escalar automáticamente entre 4 y 6 instancias EC2 según la demanda (ej. `°ALBRequestCountPerTarget`), garantizando la disponibilidad del servicio.



La Prueba del Sistema: Visibilidad Operacional con CloudWatch

Incorporamos un módulo de monitorización centralizada desplegado como parte de la infraestructura. Este panel nos permite identificar anomalías, anticipar saturaciones y validar el correcto funcionamiento del sistema.



El Motor de la Automatización: Infraestructura como Código

Todo el despliegue se realiza mediante plantillas de CloudFormation, garantizando una solución reproducible, trazable y fácil de mantener.

Estructura de Despliegue en 3 Stacks

1. "proy-icap-red"

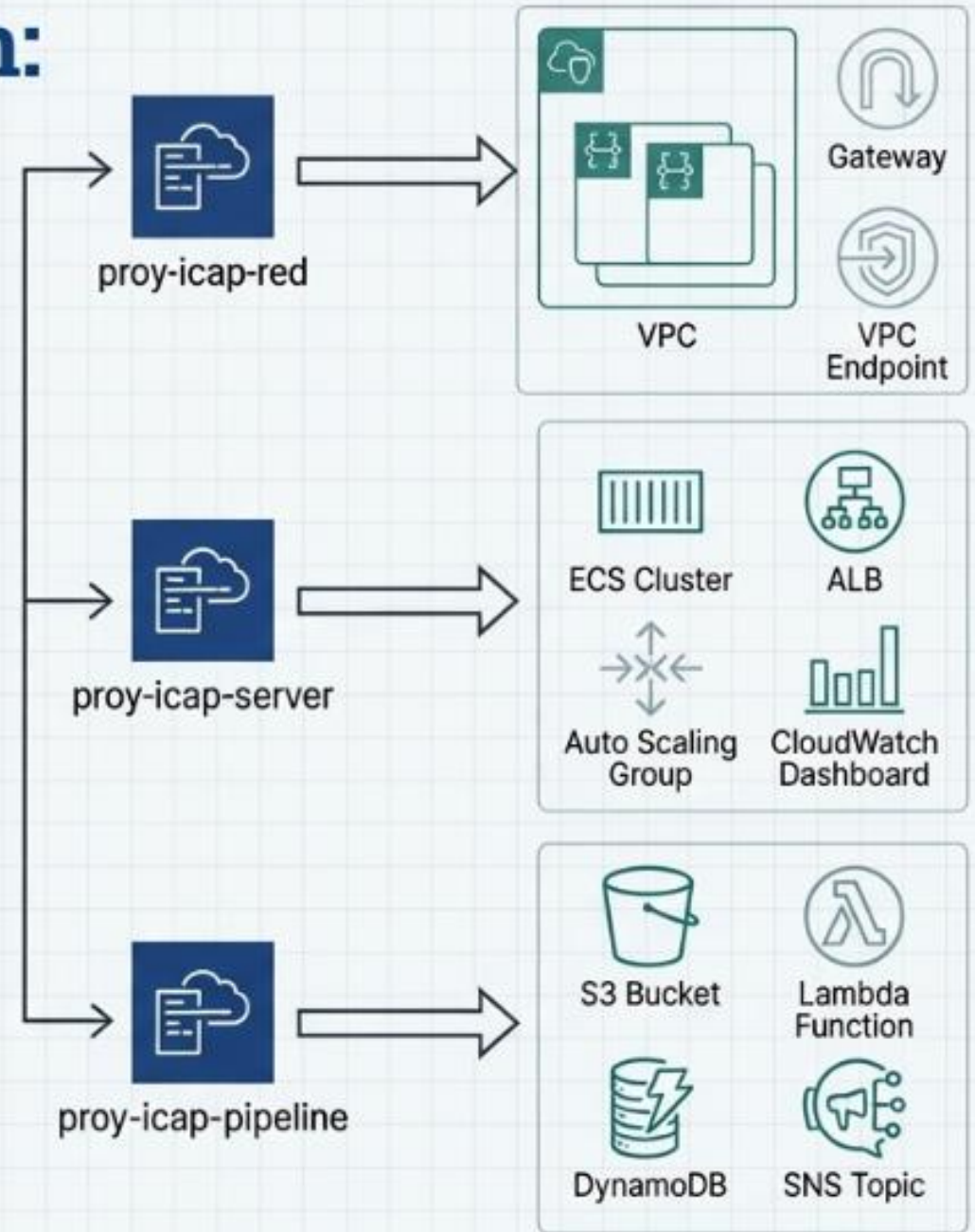
- Define la arquitectura de red base.
- (VPC, Subredes, Gateways, VPC Endpoints).

2. "proy-icap-server"

- Aprovisiona la capa de computación y servicio. (Clúster ECS, Auto Scaling Group, ALB, Dashboard CloudWatch).

3. "proy-icap-pipeline"

- Despliega el pipeline de datos serverless. (Tablas DynamoDB, Bucket S3, Funciones Lambda, Tópico SNS).



AquaSenseCloud: De un Fichero CSV a un Ecosistema de Datos Inteligente



****Resumen de Logros****

- **Automatización Completa:** Un pipeline que opera de extremo a extremo sin intervención manual.
- **Arquitectura Reactiva:** Procesamiento incremental y en tiempo real gracias a un diseño event-driven.
- **Optimización Nativa:** Uso de buenas prácticas de AWS para mejorar rendimiento y reducir costes.
- **Infraestructura como Código:** Un sistema completamente reproducible y mantenible.